

Mutual Fund Recommendation Assistant

A Project Work-I Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

BY

Akshat Malakar (EN22CS304007)

Aniket Tripathi (EN22CS304011)

Harsh Nandwal (EN22CS304027)

Under the Guidance of

Prof. Kumar Gaurav



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

January - April 2026

Report Approval

The project work "**Mutual Fund Recommendation Assistant**" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

I/We hereby declare that the project entitled "**Mutual Fund Recommendation Assistant**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering is an authentic record of our own work carried out under the supervision of the guide named in this report. The work embodied in this report has not been submitted elsewhere for the award of any other degree, diploma, fellowship, or similar distinction.

Akshat Malakar (EN22CS304007)

Aniket Tripathi (EN22CS304011)

Harsh Nandwal (EN22CS304027)

Certificate

I, Prof. Kumar Gaurav, certify that the project entitled **Mutual Fund Recommendation Assistant** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by the above-named students has been carried out under my supervision in the Department of Computer Science & Engineering, Mediacaps University.

To the best of my knowledge, the matter embodied in this report is genuine and has been prepared according to the expected academic standards for project documentation.

Project Guide
Prof. Kumar Gaurav

Acknowledgement

We express our sincere gratitude to the university administration, the Department of Computer Science & Engineering, and our guide for their academic support during the development of this project. The project benefited from continuous guidance in system design, presentation clarity, and evaluation-oriented thinking.

We are equally thankful for the availability of the local datasets, documentation, and institutional resources that enabled us to convert the project from a simple shortlist prototype into a grounded recommendation assistant with explainable outputs, corpus governance, and modular RAG-ready design.

Abstract

The Mutual Fund Recommendation Assistant is an explainable AI project designed to help investors discover suitable mutual fund schemes based on their goals, risk appetite, investment horizon, and preferred investment mode. The current implementation works over 814 schemes across 38 sub-categories and combines rule-based scheme ranking with document retrieval and grounded answer generation. Rather than behaving like a black-box chatbot, the system surfaces shortlist reasoning, caution notes, allocation guidance, and citation-backed assistant responses.

The project integrates a recommendation engine, a preprocessing and NAV-completion pipeline, a document ingestion layer, a deterministic vector index, and a citation-aware RAG assistant with advisor, auditor, and researcher modes. It is implemented using Python and Streamlit and is intentionally lightweight enough to run in an offline-friendly academic setup.

The report documents the problem context, requirements, algorithms, design diagrams, implementation choices, screenshots, observed outputs, and future expansion areas. The resulting system is positioned as an educational decision-support tool, not as a replacement for licensed financial advice.

Keywords

Mutual Fund Recommendation, Explainable AI, Streamlit Dashboard, Retrieval-Augmented Generation, Hashed TF-IDF Embeddings, Vector Retrieval, Corpus Governance, Investor Profiling

Table of Contents

Chapter 1: Introduction	13
1.1 Introduction	13
1.2 Domain Review and Project Context	13
1.3 Objectives	13
1.4 Significance	13
1.5 Research Design and Scope	14
1.6 Source of Data	14
1.7 Chapter Scheme	14
Chapter 2: Requirement Specifications	15
2.1 User Characteristics	15
2.2 Functional Requirements	15
2.3 Non-functional Requirements	15
2.4 Dependencies	15
2.5 Performance Requirements	16
2.6 Hardware Requirements	16
2.7 Software Requirements	16
2.8 Constraints and Assumptions	16
Chapter 3: Design	17
3.1 Algorithm Design	17
3.1.1 Profile-Based Recommendation Algorithm	17
3.1.2 Document Ingestion and Chunking Algorithm	17
3.1.3 Grounded Retrieval and Answer Algorithm	17
3.2 System Design	17
3.2.1 Data Flow Diagram	18
3.2.2 Use Case Diagram	19
3.2.3 Activity Diagram	19
3.2.4 Sequence Diagram	20
3.2.5 Class Diagram	20
3.2.6 Entity Relationship Diagram	21
3.2.7 Component Diagram	21
3.2.8 State Transition Diagram	22
3.3 Data and Storage Design	22
Chapter 4: Implementation, Maintenance and Testing	24
4.1 Introduction to Languages, IDEs, Tools and Technologies	24

4.2 Implementation Stages	24
4.3 Testing and Validation	24
4.4 Maintenance Considerations	24
4.5 End User Instructions	25
Chapter 5: Results and Discussion	26
5.1 User Interface Representation	26
5.2 Brief Description of Various Modules	26
5.3 Screenshots	26
5.4 Backend Representation	29
5.5 Discussion	29
Chapter 6: Summary and Conclusion	31
Chapter 7: Future Scope	32
Appendix	33
Appendix A: Example Balanced Builder Outcome	33
Appendix B: Allocation Blueprint	33
References	34

List of Figures

Figure 3.1 System Architecture Diagram	18
Figure 3.2 Data Flow Diagram - Level 0	18
Figure 3.3 Data Flow Diagram - Level 1	19
Figure 3.4 Use Case Diagram	19
Figure 3.5 Activity Diagram	20
Figure 3.6 Sequence Diagram	20
Figure 3.7 Class Diagram	21
Figure 3.8 Entity Relationship Diagram	21
Figure 3.9 Component Diagram	22
Figure 3.10 State Transition Diagram	22
Figure 5.1 Application Home and Profile View	26
Figure 5.2 Recommendation Shortlist View	27
Figure 5.3 Strategy Allocation View	28
Figure 5.4 Grounded Assistant Response View	29

List of Tables

Table 2.1 Functional Requirements Specification	15
Table 2.2 Non-Functional Requirements	15
Table 3.1 Key Project Data Assets and Outputs	22
Table 4.1 Languages, Tools and Technologies	24
Table 4.2 Validation and Test Cases	24

Abbreviations and Notations

- AI - Artificial Intelligence
- AUM - Assets Under Management
- AMC - Asset Management Company
- CSV - Comma Separated Values
- DFD - Data Flow Diagram
- ELSS - Equity Linked Savings Scheme
- LLM - Large Language Model
- NAV - Net Asset Value
- RAG - Retrieval-Augmented Generation
- SIP - Systematic Investment Plan
- UI - User Interface

Chapter 1: Introduction

1.1 Introduction

Mutual fund selection is a practical but cognitively difficult problem for everyday investors. A typical investor must evaluate return history, category risk, expense ratio, fund age, asset allocation, suitability, and minimum investment thresholds while also keeping personal goals and risk comfort in view. Many novice investors respond by over-weighting recent returns or by relying on generic category advice that lacks transparency.

This project addresses that problem through a profile-aware recommendation assistant that works with 814 local scheme records, enriched NAV and AUM data, and a small but structured knowledge corpus. The goal is not to predict market outcomes, but to make shortlist generation, reasoning, and follow-up exploration more understandable and auditable.

1.2 Domain Review and Project Context

Investor education material from AMFI, SEBI, Investor.gov, and RBI consistently emphasizes diversification, goal-based allocation, risk disclosure, and document verification before investment decisions. The project aligns with that educational framing. Instead of suggesting that one fund is universally best, the system narrows options based on suitability conditions and then supports the user with evidence-led prompts and caution notes.

The project also reflects a broader pattern in applied AI systems: structured business logic is used for high-confidence ranking tasks, while language generation is constrained by retrieved evidence to reduce unsupported claims. This makes the system more appropriate for finance-oriented academic demos than a free-form chatbot.

1.3 Objectives

- To capture investor profile inputs such as age, goal, risk appetite, horizon, investment mode, amount, and tax intent.
- To generate a shortlist of suitable mutual fund schemes with transparent fit scores and plain-language reasoning.
- To enrich scheme-level records with conservative NAV, AUM, and master-data linkage.
- To build a searchable knowledge corpus and vector retrieval layer for grounded investor guidance.
- To expose the system through an interactive Streamlit interface suitable for academic presentation and demonstration.

1.4 Significance

The significance of this project lies in its emphasis on explainability and system design discipline. The assistant does not simply rank schemes; it explains the basis of that ranking, preserves information-quality caveats, differentiates between real and synthetic NAV coverage, and uses citations when answering follow-up questions. That combination makes the project educationally stronger and closer to a serious decision-support tool.

1.5 Research Design and Scope

The work follows an iterative project design. The first phase establishes the problem statement and requirements. The second phase builds structured recommendation logic. The third and fourth phases strengthen data quality and coverage. The fifth phase adds document ingestion, local embeddings, trust-aware retrieval, and grounded answer generation. The current scope remains a decision-support prototype and intentionally avoids live execution features such as direct brokerage integration or automatic investing.

1.6 Source of Data

The project uses two principal local scheme datasets, an archive folder of historical NAV snapshots, and a managed corpus of 16 document assets. The raw scheme dataset contains 814 schemes, while the corpus layer currently exposes 43 searchable chunks after extraction and overlap-based chunking.

1.7 Chapter Scheme

Chapter 1 introduces the domain and project goals. Chapter 2 formalizes the requirements. Chapter 3 explains the algorithms, design, and diagrams. Chapter 4 discusses implementation, tools, and testing. Chapter 5 presents module behavior, screenshots, and discussion. Chapters 6 and 7 conclude the report and outline future scope, followed by an appendix and references.

Chapter 2: Requirement Specifications

2.1 User Characteristics

The primary user is a retail investor who needs a guided starting point rather than raw market data. Typical users include beginners comparing fund categories for the first time, young professionals planning long-horizon SIPs, and academically minded evaluators who want to inspect how the recommendation logic and assistant answers are grounded.

2.2 Functional Requirements

Table 2.1 Functional Requirements Specification

ID	Requirement
FR1	Capture investor inputs including age, goal, risk appetite, horizon, amount, mode, and tax preference.
FR2	Generate a ranked shortlist of suitable mutual fund schemes.
FR3	Explain why a fund fits and what the user should double-check before investing.
FR4	Provide a strategy blueprint that groups the shortlist into practical allocation roles.
FR5	Extract text from local PDF and markdown sources and convert them into searchable chunks.
FR6	Build semantic and hybrid retrieval over document chunks for grounded question answering.
FR7	Offer advisor, auditor, and researcher answer lenses inside the same UI.
FR8	Persist enriched datasets and vector index files for reuse across sessions.

2.3 Non-functional Requirements

Table 2.2 Non-Functional Requirements

Type	Expectation
Usability	The UI should remain approachable for non-expert investors and demo audiences.
Explainability	Every recommendation and answer should expose reasons, cautions, or source context.
Determinism	The current local pipeline should run without depending on live API access.
Maintainability	Core logic should remain separated into modules for data, retrieval, recommendation, and UI.
Auditability	Generated answers should retain citations and confidence labels.
Performance	Typical local interactions should feel responsive on a lightweight academic workstation.

2.4 Dependencies

- Python 3 runtime environment
- Streamlit for the dashboard interface
- pypdf for document text extraction
- Local file-system access for datasets, snapshots, and vector index storage

- A browser for launching the Streamlit application

2.5 Performance Requirements

Because the local dataset remains moderate in size, the system is designed to keep profile-based shortlist generation and grounded assistant responses within a practical interactive range. The current corpus is intentionally compact, which helps the assistant remain responsive while still demonstrating full RAG structure.

2.6 Hardware Requirements

- 4 GB RAM minimum; 8 GB recommended for smoother local experimentation
- Dual-core processor or better
- At least 2 GB free disk space for datasets, vector index files, and report artifacts
- Standard display resolution for Streamlit interaction

2.7 Software Requirements

- macOS, Linux, or Windows with Python installed
- Streamlit and pypdf as defined in requirements.txt
- A modern browser for localhost access
- Optional document tooling for future report-export or office workflows

2.8 Constraints and Assumptions

- The system is educational and must not be positioned as licensed financial advice.
- The current prototype does not depend on a live external LLM or live market feed.
- Synthetic NAV completion is used for continuity but must remain visibly distinguished from official values.
- Recommendations become stronger when the user verifies the latest scheme documents before any real decision.

Chapter 3: Design

3.1 Algorithm Design

3.1.1 Profile-Based Recommendation Algorithm

Algorithm: BuildShortlist

Input: age, goal, risk_appetite, horizon_years, amount, mode, tax_preference

Output: ranked_recommendations

1. Load scheme catalog from local dataset
2. For each scheme in catalog:
 - a. compute risk alignment score
 - b. compute goal alignment score
 - c. compute horizon alignment score
 - d. compute affordability and quality score
 - e. apply tax adjustment and style penalty
 - f. construct why-it-fits and caution text
3. Sort schemes by total score, rating, and long-horizon return signals
4. Diversify the shortlist across sub-categories
5. Return top-N recommendations with explainability fields

3.1.2 Document Ingestion and Chunking Algorithm

Algorithm: BuildCorpusChunks

Input: PDF files, markdown files, HTML/text assets

Output: document_chunks

1. Discover assets from project docs and source registry folders
2. Extract page-level text using pypdf for PDF and direct reads for text assets
3. Normalize whitespace and strip lightweight markup noise
4. Split each page into overlapping word chunks
5. Attach source metadata such as trust tier, source type, title, and page numbers
6. Return chunk catalog for lexical and vector retrieval

3.1.3 Grounded Retrieval and Answer Algorithm

Algorithm: BuildGroundedAnswer

Input: investor_question, investor_profile, shortlist, vector_index, corpus_chunks, assistant_mode

Output: answer_bundle

1. Classify query intent (fit, compare, risk, tax, strategy, or data quality)
2. Select related shortlisted schemes from question overlap
3. Build retrieval query using question + profile + scheme anchors
4. Perform semantic or hybrid retrieval over chunk vectors
5. Build citations and source cards from matched chunks
6. Synthesize answer, key points, watch-outs, follow-up prompts, and confidence label
7. Return answer bundle to the Streamlit UI

3.2 System Design

The project is structured as a modular Streamlit application. User-facing interaction is handled in the UI layer, while scheme scoring, corpus ingestion, vector retrieval, and answer generation remain separated into dedicated Python modules. This improves maintainability and also makes the design easier to explain during academic evaluation.

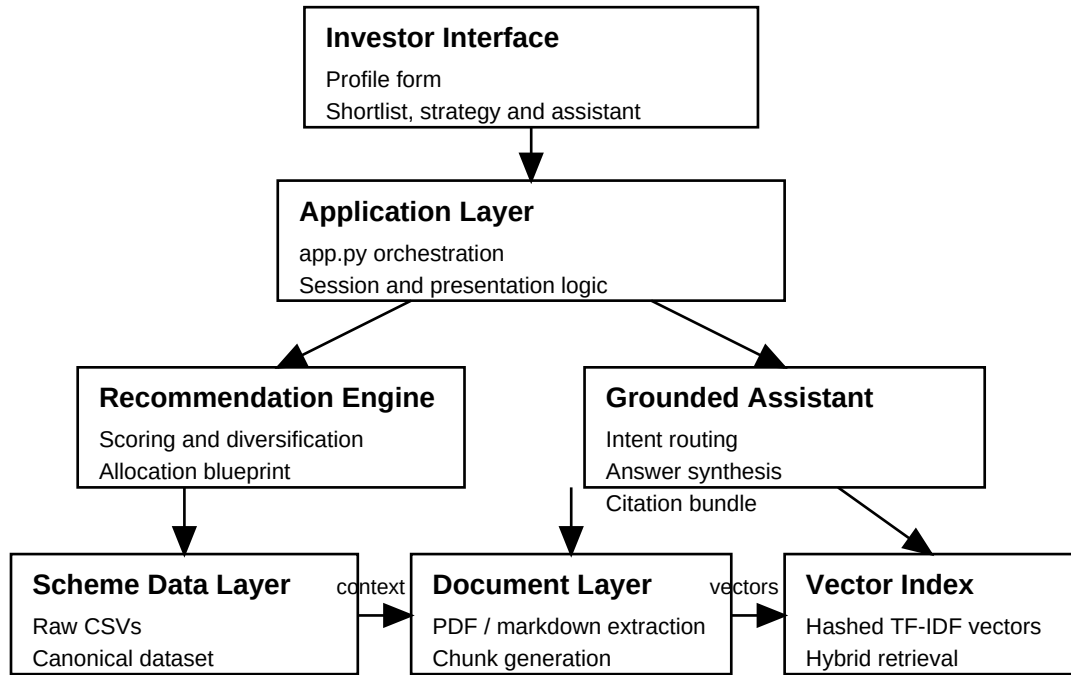


Figure 3.1 System Architecture Diagram

3.2.1 Data Flow Diagram

The level-0 data flow view treats the application as one coherent decision-support system. The level-1 view expands the same flow into profile creation, shortlist generation, strategy building, corpus retrieval, and answer composition modules.

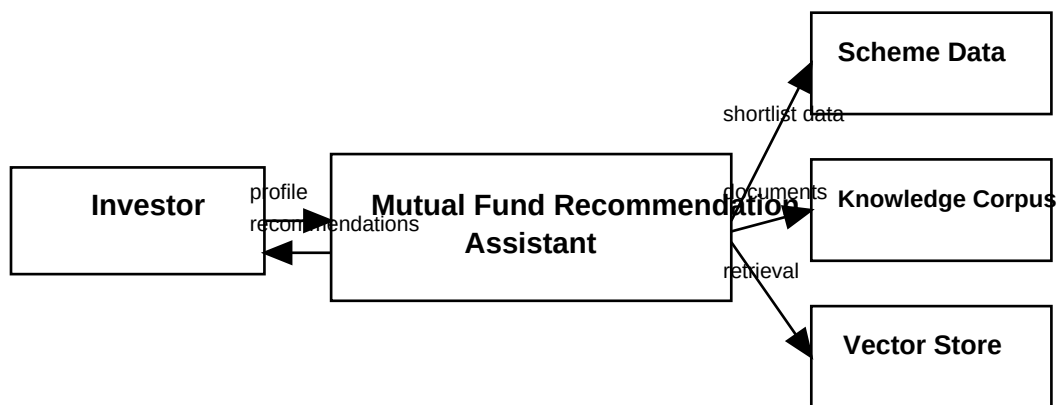


Figure 3.2 Data Flow Diagram - Level 0

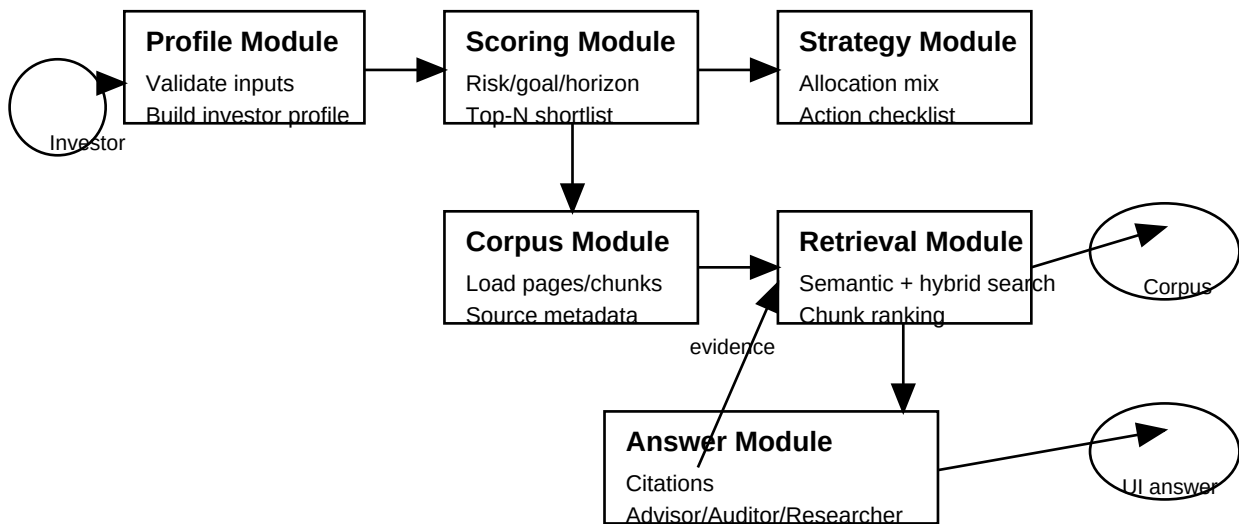


Figure 3.3 Data Flow Diagram - Level 1

3.2.2 Use Case Diagram

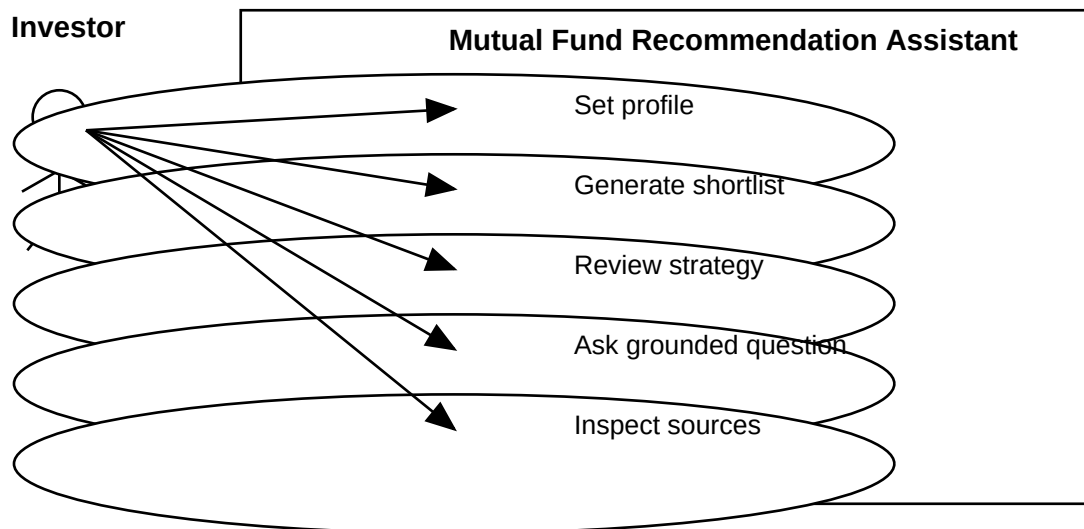


Figure 3.4 Use Case Diagram

3.2.3 Activity Diagram

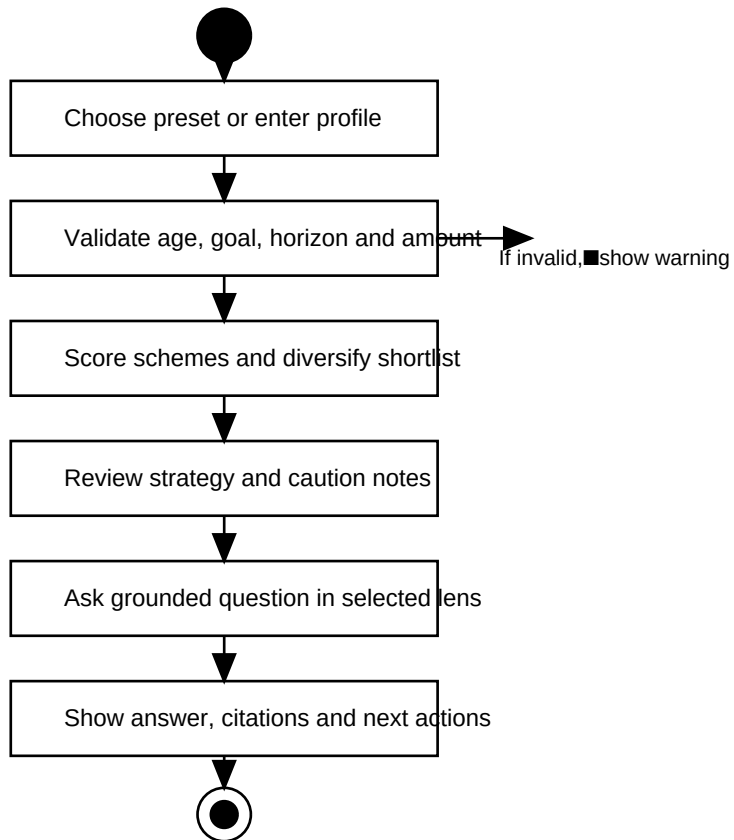


Figure 3.5 Activity Diagram

3.2.4 Sequence Diagram

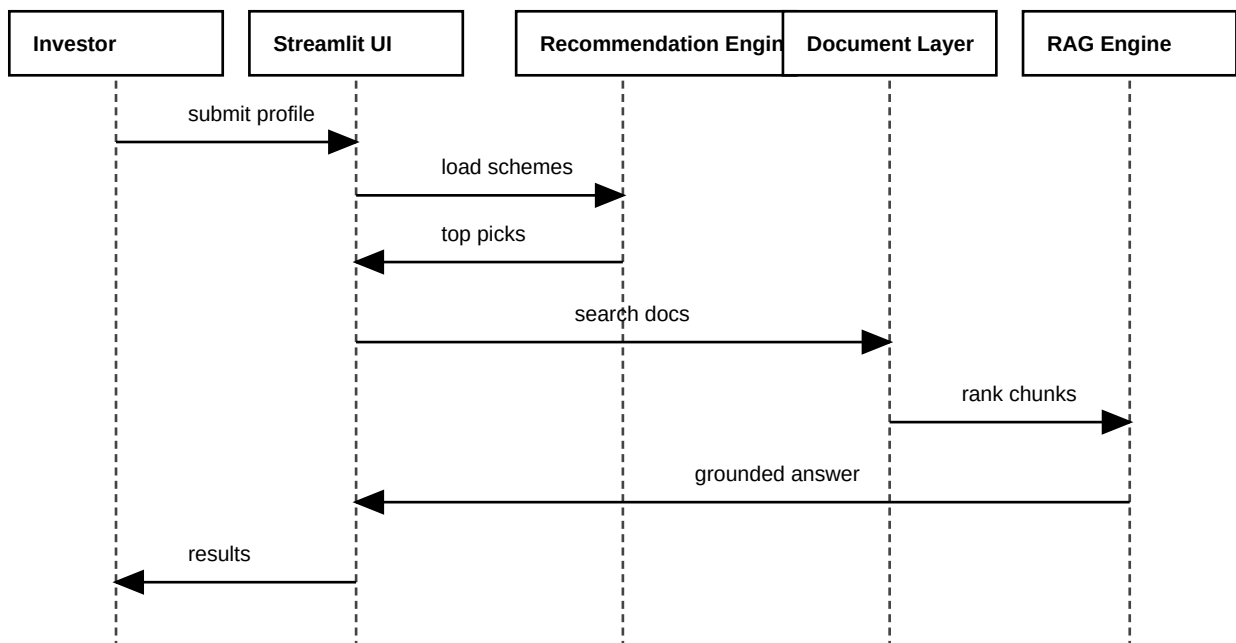


Figure 3.6 Sequence Diagram

3.2.5 Class Diagram

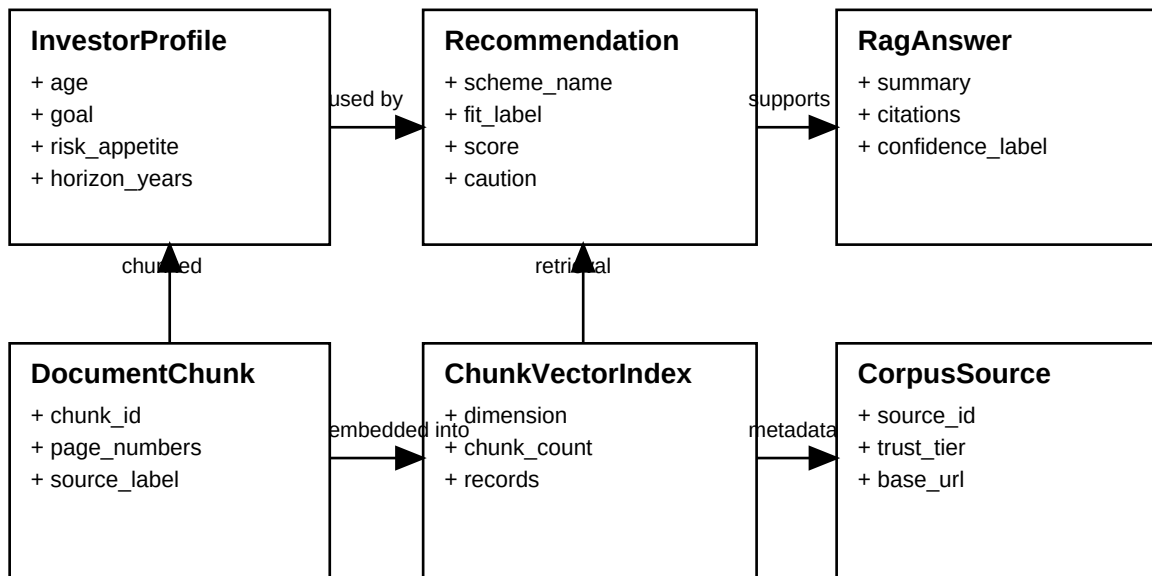


Figure 3.7 Class Diagram

3.2.6 Entity Relationship Diagram

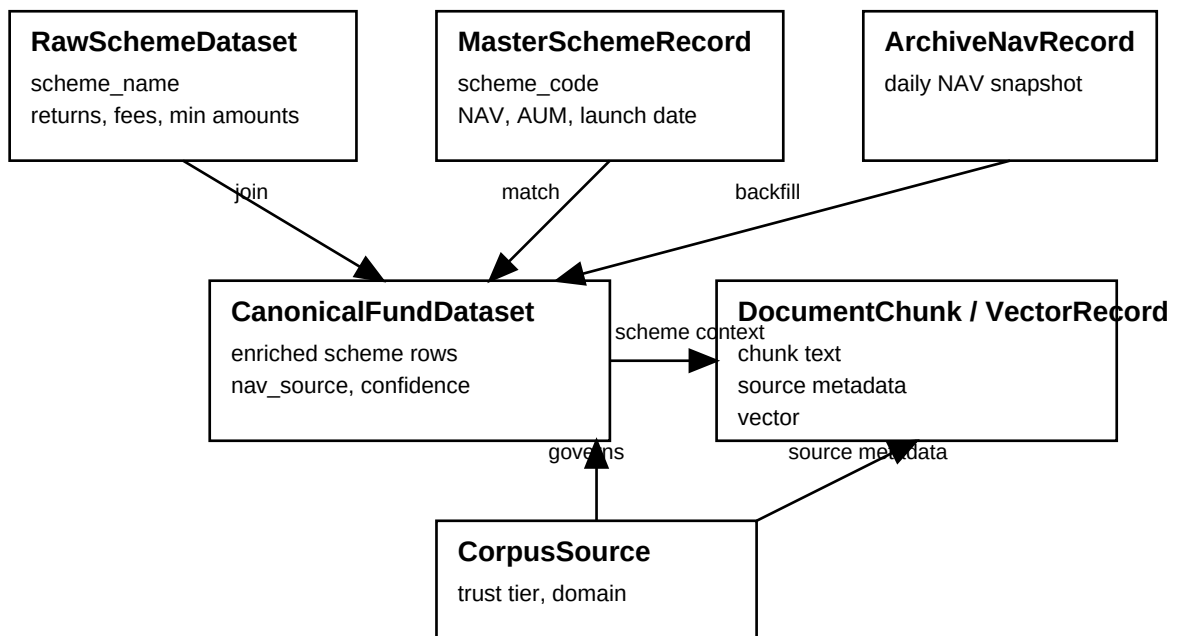


Figure 3.8 Entity Relationship Diagram

3.2.7 Component Diagram

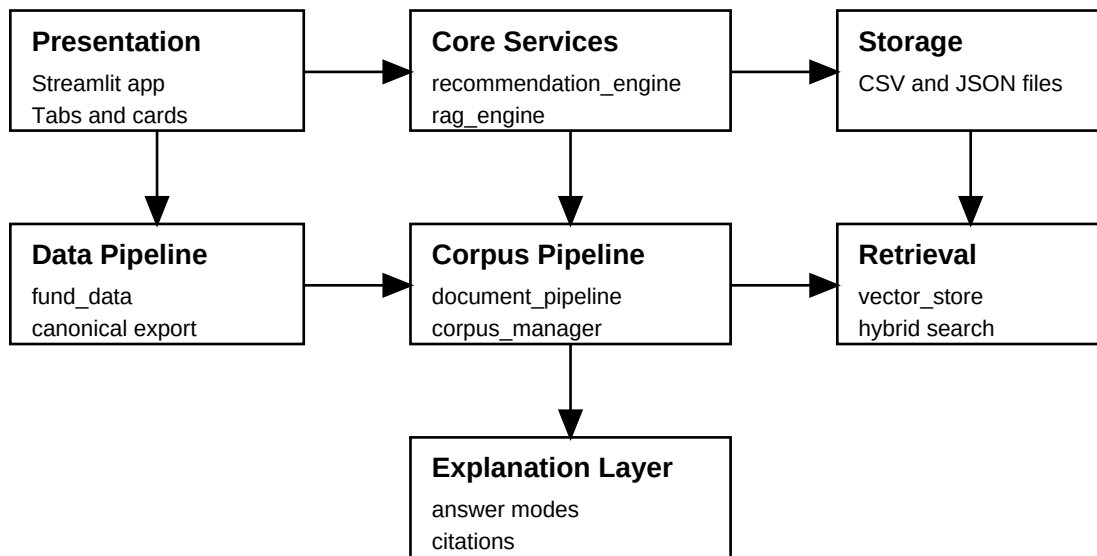


Figure 3.9 Component Diagram

3.2.8 State Transition Diagram

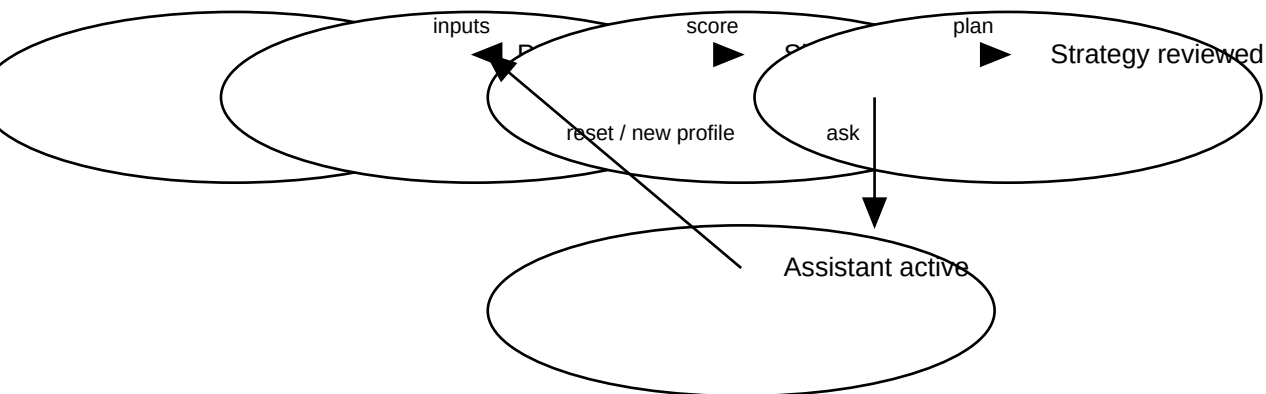


Figure 3.10 State Transition Diagram

3.3 Data and Storage Design

The project stores scheme data, enriched canonical outputs, source metadata, and vector embeddings in local file form rather than a traditional SQL database. This is a deliberate choice for portability and demo-friendliness. The current prototype uses 814 raw scheme rows, 43 document chunks, and a 384-dimension vector space.

Table 3.1 Key Project Data Assets and Outputs

Asset	Purpose	Current Detail
mutual_funds_data2.csv	Primary recommendation dataset	814 scheme rows used for shortlist scoring
mutual_fund_data1.csv	AMFI-style master dataset	NAV, AUM, scheme code, launch date, category context
archive/DailyNAV	Historical NAV snapshots	Latest detected snapshot: data_2025_01_06.csv
fund_schemes_canonical.csv	Enriched canonical export	Real NAV coverage: 504; synthetic coverage: 310
source_registry.json	Corpus governance registry	12 configured source families
document_chunk_index.json	Vector retrieval export	43 chunk vectors of dimension 384

Chapter 4: Implementation, Maintenance and Testing

4.1 Introduction to Languages, IDEs, Tools and Technologies

Table 4.1 Languages, Tools and Technologies

Technology	Role in the Project
Python	Primary implementation language for data, scoring, retrieval, and report logic
Streamlit	Interactive dashboard used for profile capture, shortlist display, and assistant interaction
pypdf	PDF text extraction for corpus ingestion
Local JSON / CSV storage	Portable storage for canonical dataset and vector index
ReportLab	Used to generate this project report in PDF form

4.2 Implementation Stages

- Foundation stage: establish the problem statement, requirement notes, and starter Streamlit interface.
- Recommendation stage: add investor profile handling, scoring rules, and shortlist explainability.
- Data stage: normalize scheme names, enrich master-data linkage, and complete NAV coverage with transparent source tags.
- Corpus stage: ingest local documents, create chunks, and attach source governance metadata.
- RAG stage: add local vector retrieval, answer modes, citations, and grounded response formatting.

4.3 Testing and Validation

The current project is validated through functional walkthroughs of the UI, recommendation output inspection, data-pipeline sanity checks, and grounded assistant verification. Since the prototype is intended for project presentation, the emphasis is on correctness of behavior, traceability of reasoning, and stability of modular interactions rather than on large-scale production benchmarking.

Table 4.2 Validation and Test Cases

ID	Scenario	Expected Result	Observed Result
TC-01	Incomplete profile input	System should request remaining fields	UI displays completion warning until mandatory values are provided
TC-02	Balanced Builder preset	Shortlist should appear with top-ranked funds	Fund recommendations are generated; top recommendations are highlighted
TC-03	Strategy tab	Allocation guidance should be visible	System shows a three-slice role-based allocation blueprint
TC-04	Assistant question with Advise mode	Grounded answer should include support and citations	Answer includes intent, confidence, citations, and source attribution
TC-05	Corpus/vector bootstrap	Searchable chunk index should be available	43 chunks and 43 vectors are available locally

4.4 Maintenance Considerations

- The codebase separates scheme scoring, corpus management, retrieval, and UI orchestration into modules for easier updates.

- The source registry centralizes trust-tier metadata so future corpus expansion remains governed instead of ad hoc.
- Canonical dataset export reduces repeated preprocessing work for downstream experiments.
- The vector index can be regenerated locally whenever the corpus changes.

4.5 End User Instructions

- Install the required packages from requirements.txt and run ``streamlit run app.py``.
- Open the local browser URL and acknowledge the risk notice.
- Use a quick preset or fill the profile form manually.
- Inspect the shortlist, strategy suggestions, and caution notes.
- Open the assistant tab, choose a lens, and ask a grounded follow-up question.

Chapter 5: Results and Discussion

5.1 User Interface Representation

The final interface organizes the workflow into three major views: shortlist generation, strategy guidance, and a grounded assistant tab. The left panel captures investor inputs and quick presets, while the right side summarizes the current spotlight and dynamic tab content. This layout makes the system easy to present in a classroom or viva setting because the profile, results, and follow-up reasoning remain visible within one window.

5.2 Brief Description of Various Modules

- Profile module: builds an InvestorProfile object from validated user inputs.
- Recommendation module: scores each scheme and produces a diversified shortlist with explanations.
- Strategy module: translates shortlist behavior into role-based allocation suggestions.
- Corpus module: extracts and chunks project and reference documents with source metadata.
- Assistant module: answers follow-up questions with intent-aware formatting and evidence citations.

5.3 Screenshots

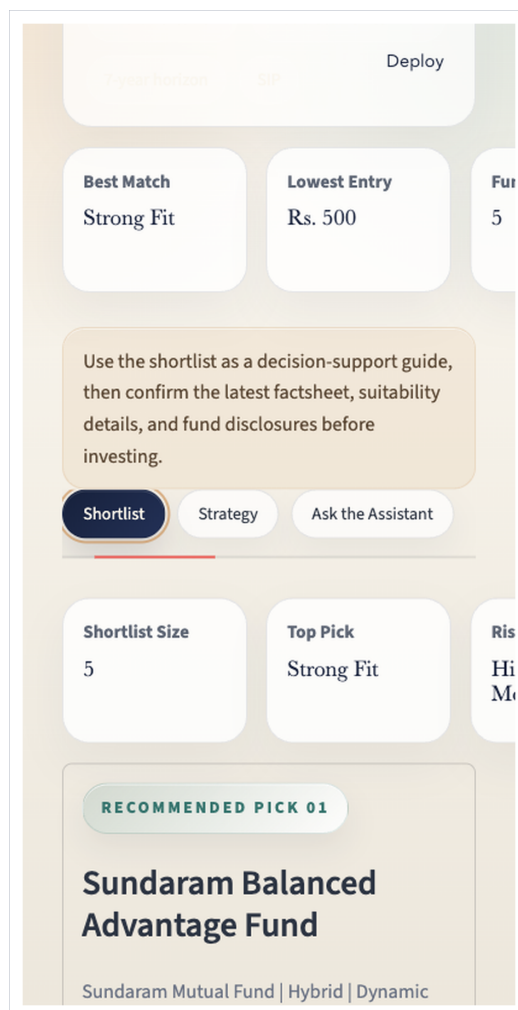


Figure 5.1 Application Home and Profile View

Deploy

814 schemes are ready for profile-based recommendations and grounded assistant guidance.

Shape Your Profile

Quick Modes

Safe Parking

Lower-risk cash and short-duration mindset.

Balanced Builder

Steady growth with controlled volatility.

Long Run

Long-horizon equity-led compounding.

Tax Focus

Tax-aware shortlist starting point.

Age

29

Figure 5.2 Recommendation Shortlist View

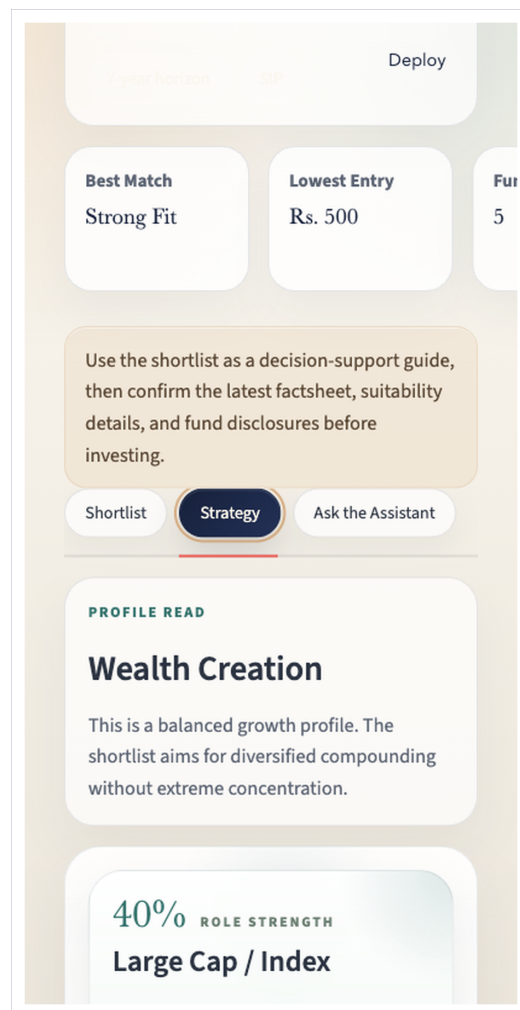


Figure 5.3 Strategy Allocation View

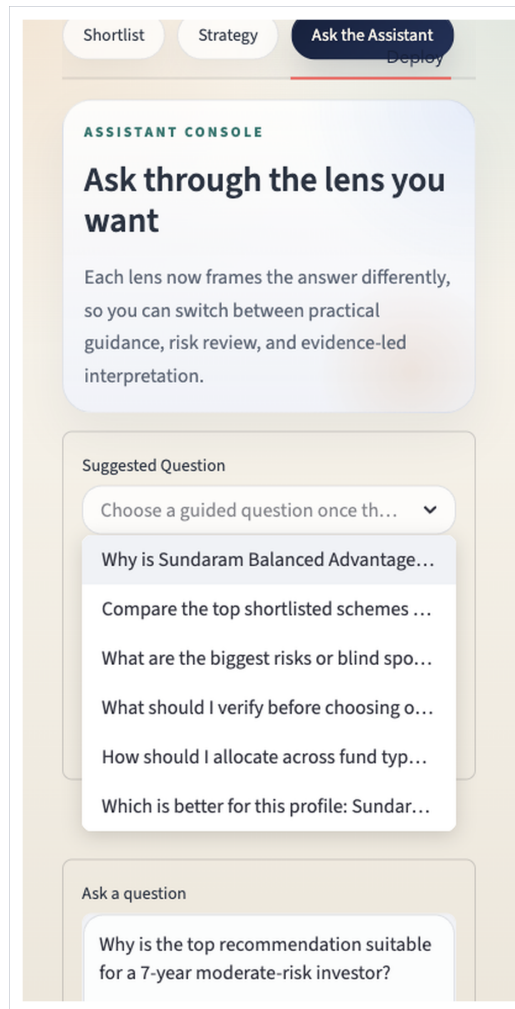


Figure 5.4 Grounded Assistant Response View

5.4 Backend Representation

- Scheme coverage: 814 schemes across 5 categories and 38 sub-categories.
- NAV enrichment: 504 rows with real master/archive support and 310 rows with synthetic continuity support.
- Corpus footprint: 16 indexed assets expanded into 43 searchable chunks.
- Vector retrieval: 43 embeddings represented in a 384-dimension deterministic vector space.
- Source governance: corpus sources carry trust-tier and source-type metadata for ranking control and explainability.

5.5 Discussion

The observed outputs show that the system behaves more like a guided decision-support assistant than a simple filter. For the representative Balanced Builder profile, the shortlist blends moderate-risk and growth-oriented funds while still surfacing caution notes for higher-risk equity entries. The assistant output is similarly structured: it reports an intent label, confidence cue, support-note count, referenced funds, and source cards.

- 1. Sundaram Balanced Advantage Fund (Dynamic Asset Allocation or Balanced Advantage) - Strong Fit, score 86
- 2. Mirae Asset Emerging Bluechip Fund (Large & Mid Cap Funds) - Strong Fit, score 81
- 3. Canara Robeco Bluechip Equity Fund (Large Cap Mutual Funds) - Strong Fit, score 81
- 4. Quant Mid Cap Fund (Mid Cap Mutual Funds) - Good Fit, score 71
- 5. Quant Flexi Cap Fund (Flexi Cap Funds) - Good Fit, score 71

The design still has clear boundaries. The corpus is intentionally modest, the vector model is local and deterministic rather than transformer-based, and the system is not intended to issue certified investment advice. Even so, the project demonstrates a meaningful integration of structured scoring, data-quality transparency, and grounded AI assistance within a coherent academic prototype.

Chapter 6: Summary and Conclusion

The Mutual Fund Recommendation Assistant combines investor profiling, explainable shortlist generation, corpus ingestion, local embeddings, and citation-aware answer synthesis into one academic prototype. The project demonstrates that a finance-oriented assistant can remain lightweight and offline-friendly while still exposing disciplined system design choices such as modular separation, trust-tagged sources, and explicit caution layers.

In conclusion, the project successfully meets its objective of making mutual fund exploration more transparent and structured for users. It does not pretend to remove the need for document verification or professional advice, but it meaningfully reduces confusion by translating investor preferences into grounded, understandable, and auditable outputs.

Chapter 7: Future Scope

- Integrate a live external LLM while preserving the current prompt-pack and citation discipline.
- Expand the corpus with more AMC factsheets, scheme information documents, and official investor-awareness material.
- Add portfolio upload and diversification diagnostics for existing holdings.
- Benchmark retrieval quality, answer faithfulness, and consistency across predefined investor scenarios.
- Introduce richer comparison views, export options, and goal-based planning workflows.

Appendix

Appendix A: Example Balanced Builder Outcome

Representative investor profile used for screenshots and result discussion: age 29, goal Wealth Creation, risk appetite Moderate, horizon 7 years, investment mode SIP, planned amount Rs. 12,000, tax-saving preference No.

Appendix B: Allocation Blueprint

Label	Allocation	Rationale
Large Cap / Index	40%	Forms a diversified core.
Flexi Cap / Large & Mid Cap	35%	Adds stronger compounding potential.
Balanced Advantage / Hybrid	25%	Helps control volatility and entry timing.

References

- AMFI India. Category framework and investor reference material. <https://www.amfiindia.com/>
- SEBI Investor Education. Investor-awareness and risk-disclosure guidance. <https://investor.sebi.gov.in/>
- Investor.gov. Diversification and asset-allocation education. <https://www.investor.gov/>
- RBI Financial Education. Financial awareness and debt-market context. <https://www.rbi.org.in/>
- Streamlit Documentation. Application framework reference. <https://streamlit.io/>
- pypdf Documentation. PDF parsing and text extraction reference. <https://pypdf.readthedocs.io/>
- Internal project documentation in README.md and docs/ within the MFRA repository.